

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B2.3.1 Construct programs that implement the correct sequence of code instructions to meet program objectives
- ▶ B2.3.2 Construct programs utilizing appropriate selection structures
- ▶ B2.3.3 Construct programs that utilize looping structures to perform repeated actions
- ▶ B2.3.4 Construct functions and modularization

B2.3.1 Sequencing

When solving problems, the order in which the statements are executed matters. Taking the swapping example in section B2.1, if you were to change the order of the instructions it would produce an incorrect output:

```
a = 5
b = 7
a = b      // 7 is stored into a
temp = a   // 7 is saved into the temporary variable temp
b = temp   // 7 (from temporary variable) is stored into b
```

Here, even if they are the same statements, the result will be incorrect just because they are not in the right order. In this example, in the end both variables, a and b, will store the value 7. As such, no swapping occurred, even if a temporary variable was introduced exactly as was done in the correct algorithm.

◆ **Sequence:** to execute instructions one after another in the given order.

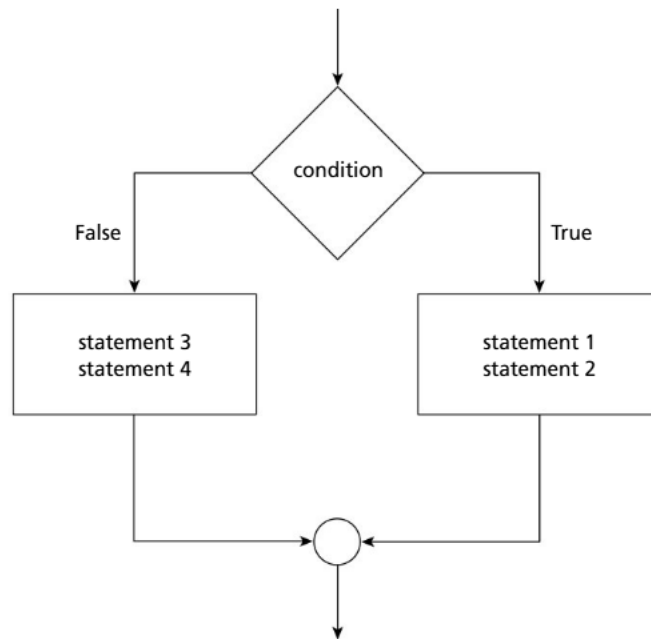
Sequence refers to the instructions being executed once and in the exact order they are written. Following the right order of the instructions is important in programming, as this impacts the outcome, which could lead to incorrect functionality (as demonstrated above), logic errors or infinite loops.

B2.3.2 Selection structures

■ Selection or branching structures

Selection is a programming construct used to decide what statement(s), if any, are to be executed based on one or more given conditions.

Conditions are usually built using logic operators. IF statements are an example of selection or branching structures. The condition in the IF statement is evaluated: if it evaluates to true, specific statements are executed; otherwise, nothing happens, or other statements are executed.



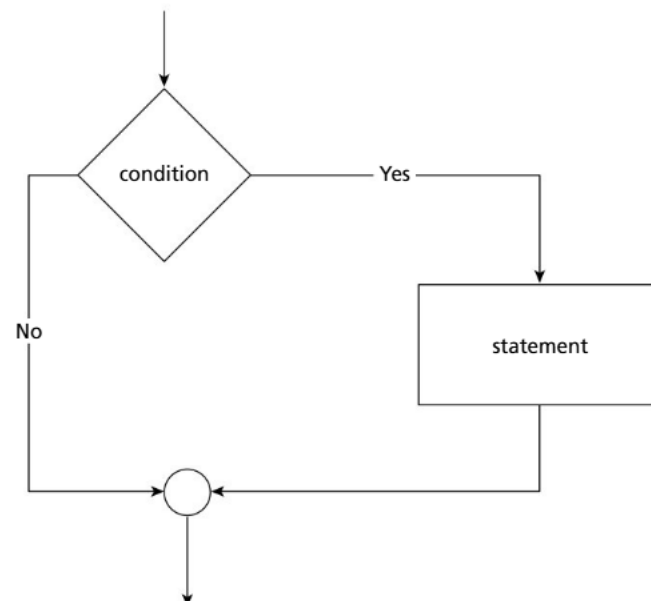
■ IF statements flowchart

In this flowchart, if the condition is met, statements 1 and 2 will be executed; otherwise, statements 3 and 4.

```

IF condition THEN
.....
END IF
  
```

■ General structure



■ Selection statement flowchart

In this case, nothing happens if the condition is not met. As an example, this structure could be used in a situation where a buyer receives a discount of 20 per cent if their purchase value is 100 or more. As such, if their purchase value is below 100, nothing happens.

Java

```
private static Scanner read;
public static void main(String[] args) {
    read = new Scanner(System.in);
    double purchaseValue, discount;
    discount=0;
    System.out.println("enter value of goods purchased: ");
    purchaseValue = read.nextDouble();
    if (purchaseValue>=100)
        discount = purchaseValue * 0.2;
    System.out.println("Your discount is: " + discount);
}
```

In this example, the `purchaseValue` and `discount` variables have been declared as `double` because they are meant to store an amount of money, which could be a decimal value. As the discount is only calculated if the condition is met, we had to initialize it to 0 in the beginning, so there is a value to display if no calculation occurs. The indentation inside the IF statement in the Python below shows what instruction is going to be executed when the condition is met. In Java, the indentation is replaced by curly brackets and, if those are missing, then the next line will be interpreted as belonging to the selection statement. Therefore, the output statement will be run independently of the given condition, as it is not included in the IF statement.

Python

```
discount = 0
purchaseValue= float(input("Enter the value of your goods:"))
if purchaseValue>=100:
    discount = purchaseValue * 0.2
print("Your discount is ", discount)
```

In Python, there is no need to declare the variables, so only the `discount` variable is initialized to 0. However, the input value is converted to `float`, to ensure the user will enter a decimal number (integers are included into the range of decimal values). If the float conversion did not happen, the user's input value would be a text, and this would throw an error when attempting to use it in calculations.

You can further expand this code and include a variable to store and display the due amount:

Java

```
double purchaseValue,toPay, discount;
toPay=0;
System.out.println("enter the value of your goods: ");
purchaseValue = read.nextDouble();
if (purchaseValue>=100) {
    discount = purchaseValue * 0.2;
    toPay = purchaseValue - discount;
}
System.out.println("You need to pay: " + toPay);
```

In the example above, you need the brackets to indicate that both calculations will happen only if the condition is met.

Python

```
toPay = 0
purchaseValue= float(input("Enter the value of your goods:"))
if purchaseValue>=100:
    discount = purchaseValue * 0.2
    toPay = purchaseValue - discount
print("You need to pay: ", toPay)
```

In Python, the indentation replaces the brackets, so they are not included.

The IF statement can be further expanded to include an ELSE. The statements corresponding to the ELSE are the statements that will be executed if the condition evaluates to false instead.

IF condition THEN

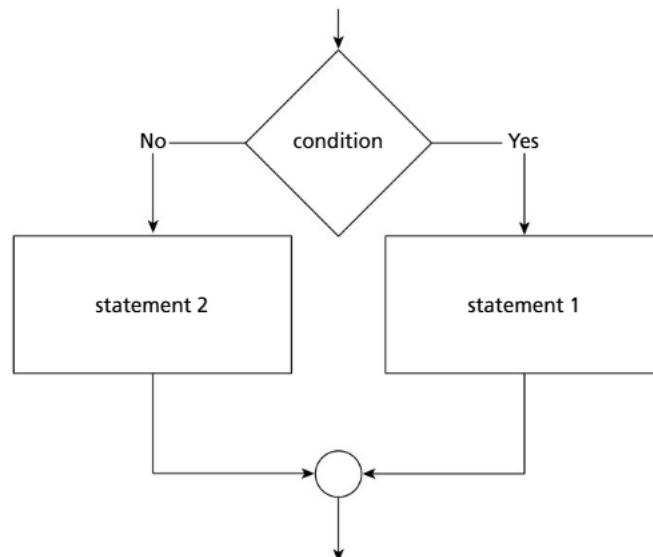
.....

ELSE

.....

END IF

■ General structure



■ Selection statement flowchart

Java

```
if (purchaseValue>=100) {
    toPay= purchaseValue * 0.8;
} else {
    toPay = purchaseValue;
}
System.out.println("You need to pay: " + toPay);
```


Note that the brackets can still be used even if only a single instruction is included in the IF or ELSE part.

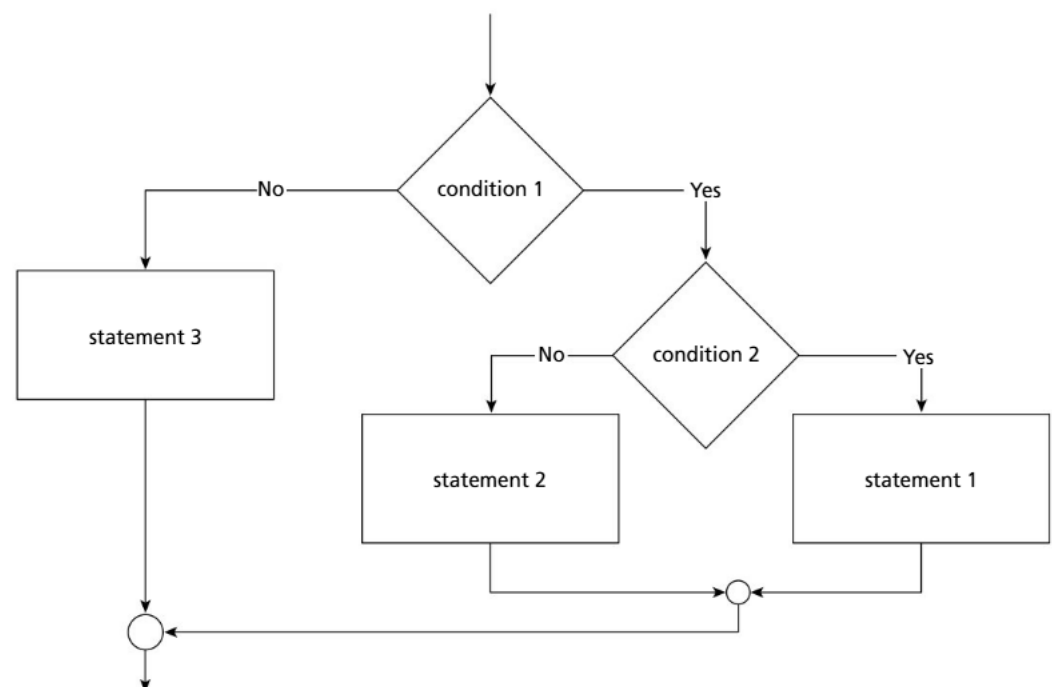
Python

```
toPay = 0
purchaseValue= float(input("Enter the value of your goods:"))
if purchaseValue>=100:
    discount = purchaseValue * 0.2
    toPay = purchaseValue - discount
else:
    toPay = purchaseValue
print("You need to pay: ", toPay)
```

As you may have realized, the block of code can even be further expanded to include another IF statement inside the current one. (One IF statement inside another is called a “nested IF”.)

```
IF condition THEN
    IF condition 2 THEN
    .....
    ELSE
    .....
    ENDIF
ELSE
    .....
END IF
```

■ General structure



■ Nested IF flowchart

Java

```
boolean member = false;
toPay=0;
System.out.println("do you have a membership card(true/
false): ");
member = read.nextBoolean();
if (purchaseValue>=100) {
    if(member == true) {
        discount = purchaseValue * 0.2;
        toPay = purchaseValue-discount;
    } else {
        toPay = purchaseValue*0.9;
    }
} else {
    toPay = purchaseValue;
}
System.out.println("You should pay: " + toPay);
```

In this case, a new variable was included that will decide whether the discount is 20 per cent or 10 per cent, depending on whether or not the buyer is a member. Also, observe that, as there is no requirement to display the discount value, you could have calculated the due amount by simply multiplying the purchase price by 0.9, thus applying the discount of 10 per cent.

Python

```
toPay = 0
purchaseValue= float(input("Enter the value of your goods:"))
member = input("Do you have a membership card (true/false)?")
member = eval(member)
if purchaseValue>=100:
    if member == True:
        discount = purchaseValue * 0.2
        toPay = purchaseValue - discount
    else:
        toPay = purchaseValue * 0.9
else:
    toPay = purchaseValue
print("You need to pay: ", toPay)
```

The value input and stored into the member variable should be a Boolean value. However, if there is an attempt to check whether the value entered is True or False by using the bool keyword instead of eval, it will not return the right answer: bool checks whether or not the string entered is empty and, as the user will write True or False, the value of the variable member will always be True as the string is not empty. Therefore, the eval function is used in this situation, to ensure the value entered is evaluated to True or False and then stored into the member variable.

However, there is another way of solving this problem, by using the and logic operator in the condition.

Java

```
if (purchaseValue>=100 && member==true) {  
    discount = purchaseValue * 0.2;  
    toPay = purchaseValue-discount;  
} else if (purchaseValue>=100 && member==false) {  
    toPay = purchaseValue*0.9;  
} else {  
    toPay = purchaseValue;  
}  
System.out.println("You should pay: " + toPay);
```

In the example above, both conditions should be met (to be a member and purchase value to be 100 or above) for the discount of 20 per cent to be applied. The second condition is checking whether both conditions are met again, but this time they should have a purchase value of 100 or above, but not be a member. The statement executed on the else branch is the statement happening if the purchase is not 100 or above.

Python

```
toPay = 0  
purchaseValue= float(input("Enter the value of your goods:"))  
member = input("Do you have a membership card (True/False)?")  
member = eval(member)  
if purchaseValue>=100 and member==True:  
    discount = purchaseValue * 0.2  
    toPay = purchaseValue - discount  
elif purchaseValue>=100 and member == False:  
    toPay = purchaseValue * 0.9  
else:  
    toPay = purchaseValue  
print("You need to pay: ", toPay)
```

In Python, else if is replaced by the keyword elif.

Conditions can use other logic operators, such as OR and NOT.

In case you wanted the discount to be applied if the purchase value was 100 or above or the buyer owned a membership card, the OR operator would replace the AND operator in the condition above, for example:

Java

```
if (purchaseValue>=100 || member)
```

Python

```
if purchaseValue>=100 or member:
```

As in the example above, a comparison with a true value can be written by using the conditional statement `if`, followed by the variable that needs to be evaluated to true:

```
if (member)
```

This will evaluate to `TRUE` if the member stores the value `true`, and to `FALSE` otherwise.

Therefore, the expression `if (! member)` in Java or `if not member:` in Python would return `TRUE` if the member variable stores a value of `false`, and it would return `FALSE` if the variable stores `true`; this expression being read as `if not member`.

Other relational operators can be part of the comparison, such as: `==`, `!=`, `<`, `>`, `<=`.

Because `=` is used in assignments, `==` is used in comparisons to check for equality.

PROGRAMMING EXERCISES

- 1 Construct code to let the user input their favourite food. If the user enters `pasta`, `PASTA` or `Pasta`, output the message "Go to Italy". Otherwise, output the message "[favouriteFood] will never replace pasta!"
- 2 Construct code that requires the user to input a number and a Boolean variable `flag`. If the `flag` is true, output double the number. Otherwise, output the squared number.
- 3 Retirement ages differ for men and women. In some countries, women can retire at the age of 62 and men at 65. Construct a program that asks the user to enter their name, gender and age. Calculate and output whether the user is eligible for retirement, or how many years they need to wait to get their pension.
- 4 Construct a program that requires the user to enter the measurements of the sides of a right-angled triangle. Calculate and display its area.
- 5 Construct code to create a calculator that converts temperatures between Celsius and Fahrenheit.
- 6 Construct code to create a quiz that asks the user to guess the capital city of five different countries of your choice. Add a score to your quiz. At the end, display the right answers for the missed questions and the score.
- 7 Construct a program that asks the user to enter their favourite colour and their lucky number. If their favourite colour is `green` and their lucky number is between 4 and 7, output the message "In [LuckyNumber] years, you'll buy a [colour] bicycle". Otherwise, if the lucky number is below 4 and above 1, output the message "I suppose you wanted a [colour] ball [LuckyNumber] years ago". Otherwise, display the message "[LuckyNumber] is not so lucky!"
- 8 Construct code that requires the user to input two numbers. Compare the two numbers and output the larger one. If the two numbers are the same, output an appropriate message instead.

TOK

How does reasoning affect decision-making?

Reasoning involves analyzing, evaluating and drawing conclusions from given facts, data and processes between them. Those are important skills in making effective decisions, as they allow you to clearly define problems and goals, by selecting the relevant details and ignoring the unnecessary ones, analysing possible solutions, generating ideas and proposing efficient solutions.

The word “decision” comes from the Latin *decidere*, meaning “to cut off”. In order to make a decision you need to consider and cut off alternative solutions. Reasoning plays a critical role in first identifying available options and then evaluating their consequences. However, decision-making often involves a blend of reasoning and other factors that may constrain rationality, such as biases and emotions.

In exploring the role of reasoning in decision-making, discuss the importance of logic, analysis, critical thinking, problem-solving techniques, acquired skills and purpose in reaching well-informed and effective conclusions.



ACTIVITY

Thinkers: Use critical and creative thinking skills to analyse and take responsible action on complex problems. Exercise initiative in making reasoned decisions.

B2.3.3 Loops and iteration

Consider a scenario where you need to display your name multiple times. Initially, you might achieve this by writing individual print statements for each repetition. However, this approach quickly becomes cumbersome when dealing with a larger number of repetitions, such as 100 times. Moreover, if you want to repeat the actions based on user input or until a certain condition is met, manually writing each repetition becomes impractical. To handle such situations efficiently, loops come into play.

Loops provide a way to execute a block of code repeatedly, either for a predetermined number of iterations or until a specific condition is satisfied. Whether you know the exact number of repetitions in advance, or need to iterate dynamically based on runtime conditions, loops offer a flexible and concise solution.

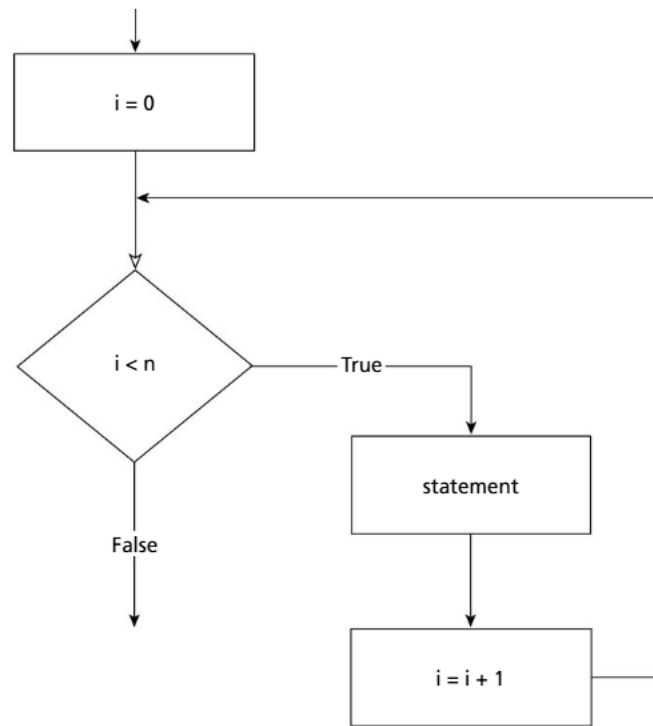
Loops are used to repeat different instructions or blocks of code. One loop refers to one repetition. There are several types of loops, such as count-controlled loops (FOR loops) and conditional loops. Conditional loops can be pre-condition loops (WHILE loops) or post-condition loops (REPEAT–UNTIL loops).

■ Count-controlled loop

A count-controlled loop (FOR loop) is used when the number of repetitions is known in advance, prior to the execution of the code. For example, if you need to write all the numbers between 1 and 100, before writing any code it can be said that the program will repeat 100 times.

```
loop i from 0 to n
.....
end loop
```

■ General structure



■ Count-controlled loop flowchart

If n is the number of repetitions required, a FOR loop would look like this:

Java

```
for(int i = 0; i < n; i++) {
    //code to be repeated
}
```

Python

```
for i in range (0,n):
    #code to be repeated
```

In Java, $i = 0$ is the starting point of the repetition, so i is initialized to 0. $i < n$ is the condition to be met so, when i is no longer smaller than n , the repetition will stop. As the variable i was initially 0 to avoid running the code to infinite, the value of i needs to be changed with every repetition. In this case, i is incremented ($i++$).

In Python, the range indicates the initial value of i (in this case, 0) and the ending value of i (in this case, n). Note here that the code will stop when i becomes $n-1$. For example, if $n = 5$, i will take the values of 0, 1, 2, 3, 4. There are five repetitions in total; when i reaches the value of 5, the repetition will stop and so the value won't be displayed.

In the example above, because the starting point is 0, which is the default value, the statement could have been written as `for i in range (n) :` and it would have had the same effect.

In both Java and Python, the starting point can be changed. Consider that you must display all the numbers between 1 and 100. To solve this problem, you can set the starting point to 1 and the ending point to 101, ensuring 100 is also included.

Java

```
for(int i = 1; i < 101; i++) {
    System.out.println(i);
}
```

Python

```
for i in range (1,101):
    print(i)
```


The Java example could have used the less-than-or-equal-to sign in the condition to ensure the upper bound of 100 is included. Alternatively, the loop could have been maintained to iterate from 0 to 100, with the printing statement adjusted to display the value of $i+1$ instead.

Java

```
for(int i = 1; i<=100; i++) {  
    System.out.println(i);  
}
```

Python

```
for i in range (100):  
    print(i+1)
```

Imagine wanting to display all the odd numbers between 1 and 100. A solution would be to keep the same structure as before, but to check whether or not the number is odd.

Java

```
for(int i = 1; i<=100; i++) {  
    if (i%2 == 1) {  
        System.out.println(i);  
    }  
}
```

Python

```
for i in range (1,100):  
    if (i%2 == 1):  
        print(i)
```

$i\%2==1$ checks whether the remainder of division by 2 of the variable i is 1. In other words, if there is a remainder, the number is odd; if there is no remainder, the number is even.

The same problem can be solved without the need to use an IF statement. A step can be included to change the incrementation so, instead of incrementing the value of i by 1, which is the default value, it increments it by a different value.

Java

```
for(int i = 1; i<=100; i=i+2) {  
    System.out.println(i);  
}
```

Python

```
for i in range (1,100,2):  
    print(i)
```

In this situation, in Java, the increment was changed, so the value of i is increased by 2, and in Python a step was included: 2, to specify the same thing. So, the variable i will take the values: 1, 3, 5, ... 99.

But what if you are requested to display the numbers from 100 to 1 instead?

Java

```
for(int i = 100; i>0; i=i-1) {  
    System.out.println(i);  
}
```

Python

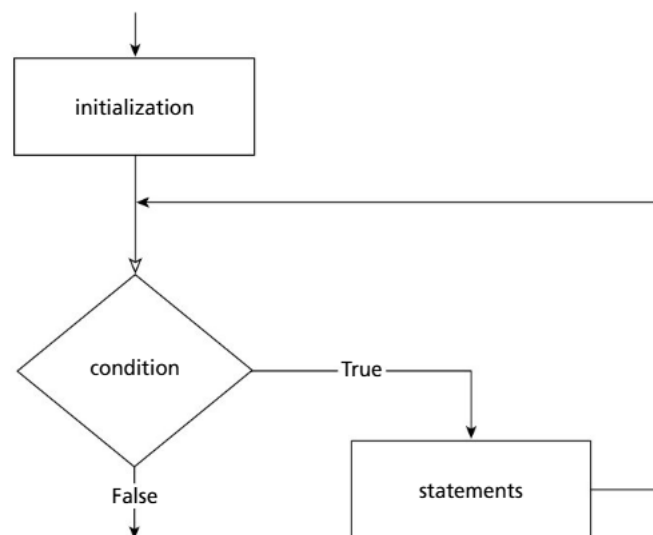
```
for i in range (100,0,-1):  
    print(i)
```

In Java, the starting point is set to 100, so that value is displayed first. The condition is changed, as you want to repeat as long as the value is greater than 0, and also the step is changed to decrement the value instead of incrementing it. $i = i - 1$ could be written as $i--$. In Python, the range is changed to start at 100. It should stop when it reaches 1 (0 is not touched) and the step is set to -1 , so the number is decremented by 1.

■ Pre-condition loop

```
i = 0
loop while
    .....
    i = i + 1
end loop
```

■ General structure



■ Pre-condition loop flowchart

A pre-condition (WHILE) loop is used when the number of repetitions is not known prior to the execution of the code. The code is expected to be repeated as long as a condition is met, or it evaluates to true.

When the condition is no longer met, called **termination condition**, the loop execution will stop. In a WHILE loop, the repetition might not run at all; this happens when the condition is never met. WHILE loops are loops that are often used for **validation** or **verification** purposes. For example, when the user is asked to enter a password twice, the computer will compare the two entries to check if they match (double entry verification). In this case, a loop would be used to repeat entering the password until the two inputs match.

◆ Termination

condition: a condition in a loop that interrupts or stops the repetition.

◆ **Validation:** a process to ensure input data is sensible or reasonable.

◆ **Verification:** a process to ensure input data is accurately copied from one source to another.

Java

```
String password, pass;
password = "apple";
System.out.println("Re-enter the password:");
pass = read.nextLine();
while (! password.equals(pass)) {
    System.out.println("Re-enter the password:");
    pass = read.nextLine();
}
System.out.println("The two passwords matched");
```

In this case, `.equals` is used to compare the value stored in `password` with the one stored in `pass`, as those two values are of type string. If the two values were numbers, then `==` would be used for comparison. The `!` is a negation, therefore, in this case, it is checking if the two values are not the same, and the user is asked to re-enter the password until the two values match. The output at the end will be displayed only after the loop is exited. If the second password never matches the first, the code inside the loop will continuously repeat.

Any FOR loop can be rewritten as a WHILE loop. Considering our example that displays all the numbers between 1 and 100, using a WHILE loop the code would look like this:

Java

```
int i = 1;
while(i<101) {
    System.out.println(i);
    i = i+1;
}
```

Python

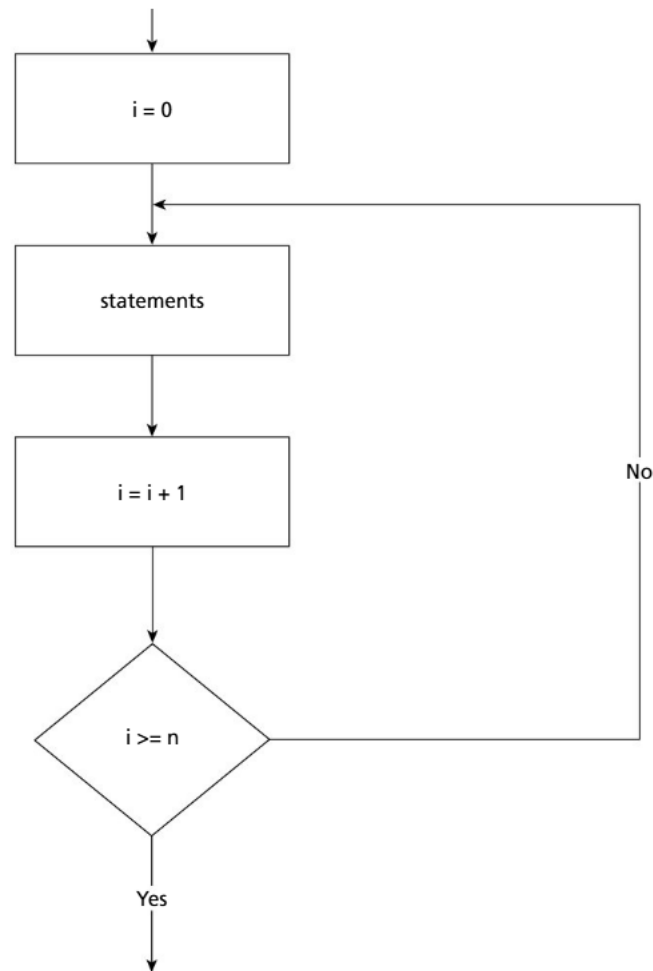
```
i = 1
while(i<101):
    print(i)
    i=i+1
```

In the example above, the loop will repeat as long as `i` is smaller than 101. The termination condition will be that `i>=101`. The role of the incrementation is to change the value of `i`, so the loop will stop at one moment. Otherwise, without that statement, the loop would run to infinite.

■ Post-condition loop

```
i = 0
repeat
    .....
    i = i + 1
until i >= n
```

■ General structure



■ Post-condition loop flowchart

In the post-condition (REPEAT–UNTIL) loop, the condition is checked at the end. In this case, the code inside the loop will execute at least once.

Both in Java and Python, post-condition loop structures are written using WHILE loops.

Java

```

int i = 1;
do {
    System.out.println(i);
    i = i+1;
} while (i<=100);
  
```

Python

```

i = 1
while True:
    print(i)
    i=i+1
    if i>100:
        break
  
```

In the example above, the variable *i* is initially set to 1. This value is displayed and then incremented. This happens no matter what the condition evaluates to. If the condition evaluates to true, the loop is repeated; if the condition evaluates to false, the repetition stops.

REVIEW QUESTIONS

- 1 Define the term “loop”.
- 2 Explain why it is useful to use loops in coding.
- 3 Explain the difference between a pre-condition and a post-condition loop.
- 4 Identify a situation when you would use a count-controlled loop.
- 5 Define the term “termination condition”.
- 6 Explain why it is important to avoid infinite loops.
- 7 Identify a situation where an infinite loop could occur.

PROGRAMMING EXERCISES

- 1 Construct a program that allows the user to enter a number. Output all numbers between 1 and that number.
- 2 Construct code that requires the user to enter a number and a word. Display each letter in the word on a separate line. Repeat the process the user’s number of times.
- 3 Construct code that asks the user to input ten numbers. Calculate and display the sum of all even numbers entered.
- 4 Construct a program that creates a variable and initializes it to the value 7. Ask the user to guess a number between 1 and 20. If the user guesses the value 7, output the number of attempts. Do not allow the user to try to guess the number more than three times. If the user doesn’t guess the number after three attempts, output the message: “The game is locked. Try again later!”
- 5 Construct a program that asks the user whether they want an apple. Repeat the question until their answer is yes. At the end, output the message: “Apples are delicious, aren’t they? Let’s have one!”

ACTIVITY

Use your answers to the programming exercises above to answer the following questions.

- 1 What loop did you use to solve question 1? Could you replace that loop with another one? How would you do that?
- 2 Did the program you constructed to solve question 2 handle user input correctly? Can you think of an easier way to extract each letter from a given word?
- 3 Did your program for question 3 extract the even numbers correctly? What change would calculate the sum of all the odd numbers instead?
- 4 Why do you think a FOR loop is not appropriate for solving question 4?
- 5 What termination condition did you use for question 5?

Key information

FOR loops are called “count-controlled” loops, WHILE loops are called “pre-condition” loops and REPEAT-UNTIL loops are called “post-condition” loops.

Count-controlled loops are used when the number of repetitions is known prior to the execution of the code. Pre-condition loops might not be executed at all (if the condition is not met). Post-condition loops will execute at least once, even if the condition is never met.

Top tip!

When you are asked to identify the type of loop exemplified in a piece of code, refer to them as count-controlled, pre-condition or post-condition loops. For example, saying it is a FOR loop instead of a “count-controlled” loop is not enough, as it is not clear that you know what type of loop a FOR loop is.

B2.3.4 Functions and modularization

◆ **Maintainable code:**

clear, easy-to-read and modify code that can be reused within the same program or in other programs, by the same or other programmers.

◆ **Procedure:** a set of statements that can be grouped together and called in a program as needed; they don't return a value.

◆ **Function:** a set of statements that can be grouped together and called in a program as needed; they always return at least one value.

Functions represent blocks of code that can be reused in various parts of the program and include a return value. Using functions makes the code look neater, and it saves development time as they can be reused as needed. At the same time, they allow for modularity, meaning multiple programmers can work together, each developing a function and then those functions can be put together to build up a more complex program. Once a function has been written and tested, the programmer can reuse it without worrying about its functionality.

Writing **maintainable code** is an important skill for any programmer. Maintainable code refers to code that is clear; easy to read and modify; and can be reused by the programmer themselves or by other programmers. Using functions aids in the creation of maintainable code.

Such a reusable block of code is called a **procedure** or a **function**; the difference between the two being the returned value(s). A procedure can take several parameters or none and it has no return value, so it could update a variable or output a message or a value. However, a function will return a value. This returned value can further be used in a mathematical expression, or it can simply be displayed.

Consider the following problem: A user is required to provide two whole numbers. Construct a program that calculates the sum of the two numbers and displays it.

A simple solution would be to ask the user to enter the two numbers, add them together and display the result, such as:

Java

```
read = new Scanner(System.in);
int no1, no2, sum;
System.out.println("enter no1:");
no1 = read.nextInt();
System.out.println("enter no2:");
no2 = read.nextInt();
sum = no1+no2;
System.out.println("The result is "+ sum);
```

Python

```
no1 = int(input("enter no1:"))
no2 = int(input("enter no2:"))
sum = no1 + no2
print("The result is ", sum)
```

■ Solution 1

But if you needed to perform the same operation again, with different values, you would have to rewrite the code, which is not very time or memory efficient. Therefore, to complete such tasks, a procedure is required.

Java

```
private static Scanner read;
public static void add() {
    int no1, no2, sum;
    System.out.println("enter the first number:");
    no1 = read.nextInt();
    System.out.println("enter the second number:");
    no2 = read.nextInt();
    sum = no1+no2;
    System.out.println("The result is "+ sum);
}
public static void main(String[] args) {
    read = new Scanner(System.in);
    add();
}
```

Python

```
def add():
    no1 = int(input("enter no1:"))
    no2 = int(input("enter no2:"))
    sum = no1 + no2
    print("The result is ", sum)
add()
```

■ Solution 2

Because this does not return the sum, but only prints it, it is a procedure.

Java

```
private static Scanner read;
public static void add(int no1, int no2) {
    int sum;
    sum = no1+no2;
    System.out.println("The result is "+ sum);
}
public static void main(String[] args) {
    read = new Scanner(System.in);
    int value1, value2;
    System.out.println("enter the first number:");
    value1 = read.nextInt();
    System.out.println("enter the second number:");
    value2 = read.nextInt();
    add(value1, value2);
}
```

Python

```
def add(no1,no2):  
    sum = no1 + no2  
    print("The result is ", sum)  
value1 = int(input("enter no1:"))  
value2 = int(input("enter no2:"))  
add(value1, value2)
```

■ Solution 3

In the code above, the same procedure is created, but two parameters are passed to it (no1 and no2). When the procedure is called in the main program, the values being passed to it are called “arguments” (value1 and value2).

When a return is added to the block of code, the procedure becomes a function. The returned value can be displayed on the screen or used in other mathematical expressions. The number of parameters also differs when it comes to functions: one, none or more parameters can be passed to a function.

When implementing functions in Java, the void keyword from the method signature is replaced by the data type of the value being returned.

Java

```
private static Scanner read;  
public static int add(int no1, int no2) {  
    int sum;  
    sum = no1+no2;  
    return sum;  
}  
public static void main(String[] args) {  
    read = new Scanner(System.in);  
    int value1, value2;  
    System.out.println("enter the first number:");  
    value1 = read.nextInt();  
    System.out.println("enter the second number:");  
    value2 = read.nextInt();  
    System.out.println("The result is "+ add(value1, value2));  
    int newVal = 3;  
    newVal = newVal + add();  
    System.out.println("The new value is "+newVal);  
}
```

Python

```
def add(no1,no2):  
    sum = no1 + no2  
    return sum  
value1 = int(input("enter no1:"))  
value2 = int(input("enter no2:"))  
print("The result is ", add(value1, value2))  
newVal = 3  
newVal = newVal + add(value1, value2)  
print("The new value is ", newVal)
```

In the example above, the function `add` will return a value of data type integer. The function is called once to be displayed and the other time to increase the value of the variable `newVal`.

REVIEW QUESTIONS

- 1 Compare functions and procedures.
- 2 Identify three things you can do to keep a maintainable code.
- 3 Outline the difference between validation and verification. Provide an example where a selection can be used for validation checks.
- 4 Differentiate between pre- and post-condition loops.

PROGRAMMING EXERCISES

- 1 Construct code to let the user create a function called `factorial` that takes a natural number (`n`) as a parameter. The function should return the factorial value of that number (`n!`).
 - 2 Construct a program that consists of a procedure called `odds` that takes a natural number (`n`) as a parameter. The procedure should display all the odd numbers from 1 to that number (`n`).
 - 3 Construct code that creates a function called `perfect` that takes `n` (a natural number) as a parameter. The function should return `true/false` if `n` is a perfect number.
A perfect number is a positive integer that is equal to the sum of its positive proper divisors, excluding the number itself. For example, 28 is a perfect number as its proper divisors are: 1, 2, 4, 7, 14. Adding those values together we get 28, and this result is equal to the initial number.
 - 4 Construct a function called `palindrome` that takes a word as a parameter. The function should return `true/false` if the given word is a palindrome.
A palindrome is a word that reads the same forwards or backwards. For example, kayak, racecar, level and civic are all palindromes.
- Reflect on the errors you encountered when constructing your code, and how you resolved them.
- How did you go about debugging the issues?
 - Did you notice any patterns in the errors you encountered?
 - What can you do to avoid such errors in the future?

◆ **Local variable:** a variable that exists only within the block of code where it is defined.

◆ **Global variable:** a variable that exists throughout a program.

◆ **Variable scope:** the lifetime of a variable within a program; it determines whether you can access and modify the variable within a specific block of code.

■ Scope of variables

Based on their scope, variables can be classified into various categories, such as **local** or **global** variables.

The **scope of the variable** defines its lifetime in the program, meaning the block of code where it has been declared, where it can be used and modified. How and where the variables are declared defines the scope.

Local variables

Local variables are variables that have their scope limited to the block of code within which they are declared and used.

Once that block of code is executed, the variable is automatically removed from the memory. It is recommended to use local variables as often as possible. An example of a local variable is when using a counter in a FOR loop. That counter is required only within the body of the loop for storing temporary data but, once the loop is finished, you do not need that counter any longer.

Java: *i* is a local variable

```
for(int i=0; i<3;i++) {  
    System.out.println("hello world");  
}
```

As *i* is a local variable, it exists within the loop, and it will take values from 0 to 3. When it reaches 3, the variable is no longer smaller than 3 and the repetition will stop. After this loop, the variable *i* does not exist any longer.

Python: *text* is a local variable

```
def hello():  
    # text is the local variable  
    text = "hello world"  
    print(text)  
# code to text  
hello()  
# the print below will throw an error as text does not exist  
# outside the procedure  
print(text)
```

In this example, *text* is the local variable. When printing the variable *text* inside the subprocedure, the message “hello world” will be displayed. However, when attempting to print the *text* again, after the call of the method, an error will be thrown as *text* is used without having an initial value.

Global variables

Global variables are variables that are visible and accessible throughout the program.

In Python, the global variables are initialized at the top of the code or module and, whenever they are used within a function or procedure, they are declared as global. As Java is an

object-oriented programming language, it does not use the concept of global variables. However, by using the `static` or `static final` keywords, the variables can have all the properties of global variables.

Python

```
text = "hello "
def hello():
    # text is a global variable
    global text
    # text is concatenated with a new text value
    text = text + "world"
# Driver code
hello()
print(text)
```

In this example, the `text` is a global variable, which means that, by calling the `hello` procedure, the content of the variable `text` is changed and, once it is printed, even if it is printed outside the procedure, it will display the updated content: `hello world`.

Top tip!

When writing code, aim to preponderantly use local variables rather than global variables. Local variables are confined to the function or block in which they are declared, promoting encapsulation and modularity. This makes functions self-contained and easier to understand, test and debug. Global variables can be modified by any part of the code, leading to unintended side effects that can make the program behaviour difficult to predict and debug. Local variables prevent such side effects by limiting the scope of variable modifications.

REVIEW QUESTIONS

- 1 Trace the following program to determine its output.

Java

```
static String student = "Bob";
public static void changeOfName(String st) {
    st = "Jim";
    System.out.println("Student inside the method: "+st);
}
public static void main(String[] args) {
    changeOfName(student);
    System.out.println("Student outside the method: " +
        student);}
```

Python

```
student = "Bob"
def changeOfName(st):
    st = "Bobby"
    print("Student inside the method: ", st)
changeOfName(student)
print("Student outside the method: ", student)
```

- 2 Identify a local variable in the code above. Explain why the chosen variable is not a global variable.
- 3 Construct a program that changes the code above so that, after the method is executed, both outputs display the name Jim, without changing the first line of code.
- 4 Construct code to update the method above to include a validation check: the new name should be input from the keyboard and the change should occur only if the new name is different from the previous name. Display a message if the new name is not different and display the current name.
- 5 Identify the scope and data type of the variable you have created to validate the name change.

How can we apply programming to solve problems?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B2.1.3 Describe how programs use common exception handling techniques
- ▶ B2.1.4 Construct and use common debugging techniques

B2.1.3 Exception handling

When a computer program is run, different reasons or events might cause the program to halt or produce an unexpected outcome. These reasons could include logical errors in the code, unexpected user inputs or resource unavailability.

◆ **Logic error:** an error in a program that makes it operate incorrectly; it will not crash the program.

◆ **Runtime error:** an error that occurs when executing a program; the program might stop unexpectedly.

◆ **Exception handling:** a process of responding to an exception, so the system does not halt unexpectedly.

◆ **Exception:** an unexpected event that stops the execution of a program, e.g. division by 0.

Logic errors refer to incorrect sequences in the logic of the program, incorrect choices of condition or incorrect calculations. For example, when attempting to calculate the average of three numbers, dividing the sum of the three numbers by 2 instead of dividing it by 3 would produce an incorrect outcome.

Such errors can only be detected through testing, as they would pass the compilation stage.

Runtime errors could cause the program to crash. These errors refer to problems occurring as the program runs, such as division by 0; a file not being found; truncation; overflow or underflow errors; a hardware device not being available, such as a printer not being ready; or a class file not being found.

Also, a user can enter unexpected inputs, such as entering text instead of a number or entering values that would lead to an attempt to divide by 0, or entering the wrong file location to read data from or write data to.

Resource unavailability refers to hardware and software equipment not being available for the operation, such as a file not being found or a printer not being ready for the operation.

All those events can be dealt with, so the program does not crash, using **exception handling**. Even if the desired operation would not be achieved, the user can get an idea of what went wrong, and they can continue attempting other features of the program.

The role of exception handling techniques is to maintain the normal flow of the program, by catching and throwing **exceptions** that cannot be handled locally. In Java, this takes the form of `try/catch` blocks, while in Python they are found as `try/except` blocks. The code that might throw an error is written within the `try` block and the exception is caught and displayed, if needed, in the `catch/except` block. Both languages allow for a `finally` block that is found at the end of the `try/catch` or `try/except` block and this includes code that will always execute after leaving the `try` statement, regardless of the `try` block outcome: whether or not it raises an error.

Java

```
read = new Scanner(System.in);
System.out.println("Enter a number: ");
int number = read.nextInt();
int result;
try {
    result = 10/number;
    System.out.println(result);
}
catch(Exception e){
    System.out.println(e.toString());
}
finally {
    System.out.println("This would be printed anyway");
}
```

Python

```
number = int(input("Enter a number: "))
try:
    result = 10/number
    print(result)
except ZeroDivisionError:
    print("You can't divide by zero")
finally:
    print("This would be printed anyway")
```

In the example above, the user is requested to enter a number. If the number entered is 0, then this would be caught in the exception; otherwise, the calculation will be performed and the result displayed. Independent of the action completed, the message included in the `finally` block will be displayed.

Java

```
read = new Scanner(System.in);
System.out.println("Enter a number: ");
int number = read.nextInt();
int result;
try {
    result = 10/number;
    System.out.println(result);
}
catch(Exception e){
    System.out.println(e.toString());
}
```

Python

```
number = int(input("Enter a number: "))
try:
    result = 10/number
    print(result)
except:
    print("There was a problem!")
```

The exception handling construct can include only a `try/catch` block in Java, or just a `try/except` block in Python. There is no need to specifically indicate the type of error that might have caused the program to crash, but this is useful to help the programmer debug the code and fix issues that might be solved or to let the user understand the problem if the wrong input is provided.



Key information

"Exception" refers to the event that interrupts the execution of a program, while "exception handling" refers to the actions taken to deal with an exception or how the system is prevented from halting unexpectedly. For example, a division by 0 is an exception; using a `try/catch` or `try/except` block is the exception handling technique.

REVIEW QUESTIONS

- 1 Explain how exceptions differ from errors.
- 2 Explain why it is important to handle exceptions in your code.
- 3 Identify some common types of exceptions.
- 4 Explain a scenario when a runtime error could occur.
- 5 Explain the risks of showing detailed exception messages to end users.

B2.1.4 Debugging techniques

◆ **Debugging:** finding and fixing errors in a program.

◆ **Trace table:** a technique used to test an algorithm, and to predict how it will be run and how values of variables will change.

◆ **Breakpoint:** a marker to interrupt the execution of code for debugging purposes.

Debugging refers to finding and fixing errors in code. Common debugging techniques include **trace tables**, **breakpoint** debugging, print statements and step-by-step code execution.

Trace tables

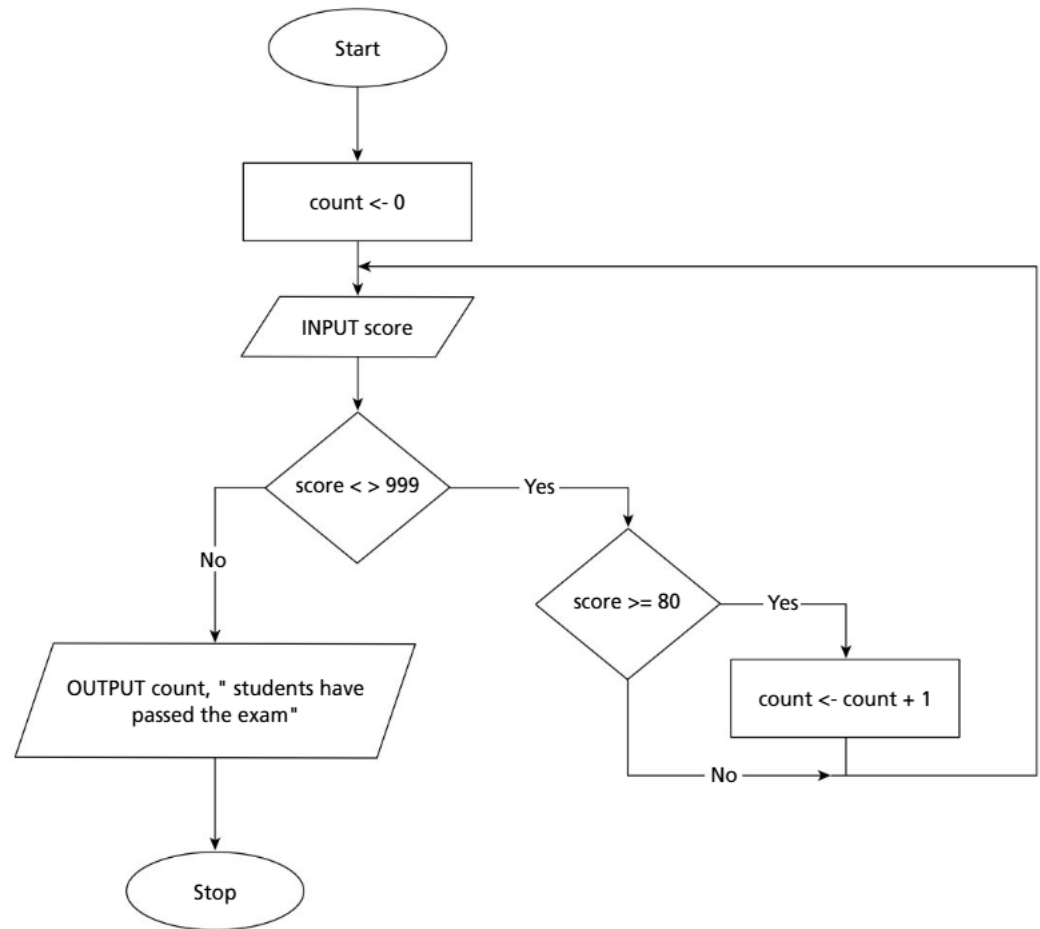
Trace tables represent a technique usually used at design stage to test an algorithm and predict step by step how it will run. They can be used to demonstrate the outcome of an algorithm or to identify logic errors.

A trace table is a table in which the columns represent variables, conditions or an output in the algorithm. However, not all variables, conditions or outputs are always needed; this depends on the purpose of the trace table.

Its role is to identify how variables change, what conditions evaluate to and what are the produced outcomes. By producing a trace table, the purpose of the algorithm can be determined or any flaws in the algorithm can be detected.

For example, consider the following problem: Students taking a language course will pass their final examination if their score is 80 or above. As the students register for the course every term, the number of students is unknown. Therefore, the teacher will enter 999 to terminate the program. The teacher is required to identify the number of students passing the assessment.

The following flowchart has been designed to suggest a possible solution to the problem:



The trace table for the input data would look like this:

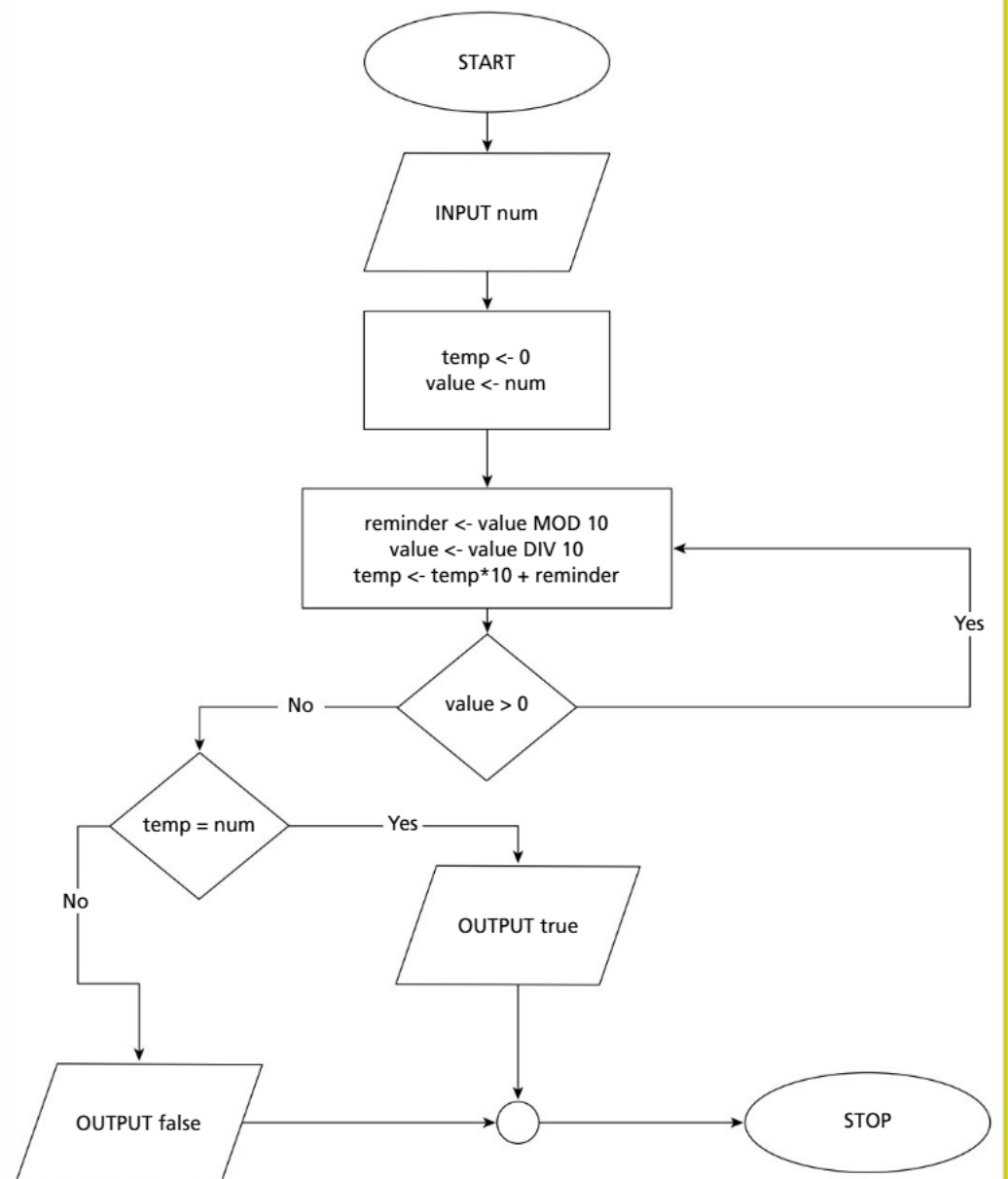
23, 98, 33, 45, 78, 80, 81, 84, 34, 999

count	score	output
0	23	
	98	
1	33	
	45	
	78	
	80	
2	81	
3	84	
4	34	
	999	4 students have passed the exam

When the value 98 is entered, the count is incremented. 33, 45, 78 will not affect the count, so its value stays the same. You might choose to repeat the previous value for the count variable, or just leave it blank as there is no change. Be aware if there is a statement reassigning the value of the variable to something, then even if the value is the same as the previous one, it should appear within the count column, as that is a change to the variable. When value 80 is entered, the count is incremented again, and the procedure repeats for the values 81 and 84, but nothing happens to the count when 34 is entered. 999 is the value that will terminate the program, and so the output will be displayed, as the output is displayed only after 999 is entered, in this example.

REVIEW QUESTIONS

The following flowchart represents an algorithm.



- 1 Copy and complete the trace table below for the value: 1221.

num	temp	value	reminder	value > 0	temp = num	output

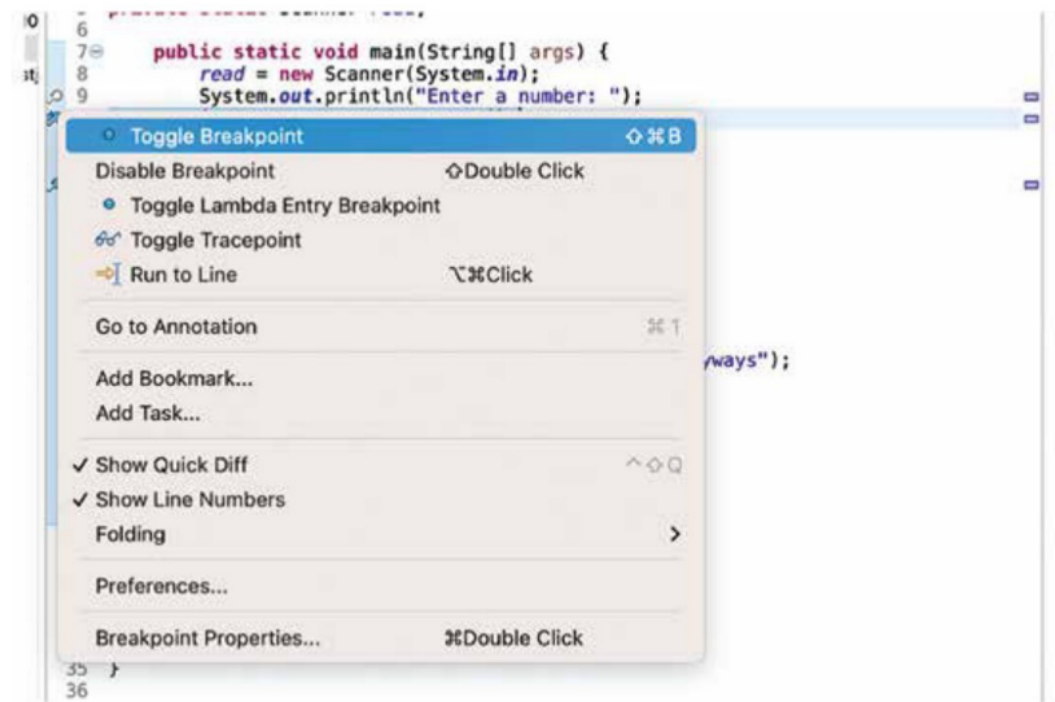
- 2 Copy and complete the trace table below for the value: 1231.

num	temp	value	reminder	value > 0	temp = num	output

- 3 Identify the purpose of the algorithm represented by the flowchart above.

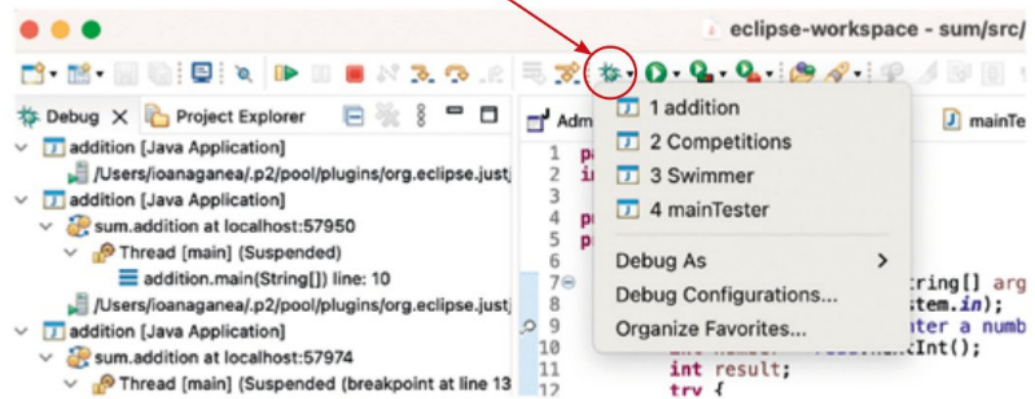
■ Breakpoint debugging

Breakpoints are special markers that interrupt the execution of the code for debugging purposes. To set a breakpoint in Eclipse IDE, it is sufficient to right-click on the blue section beside the line number and toggle a breakpoint as shown in the screenshot below (this differs from IDE to IDE).



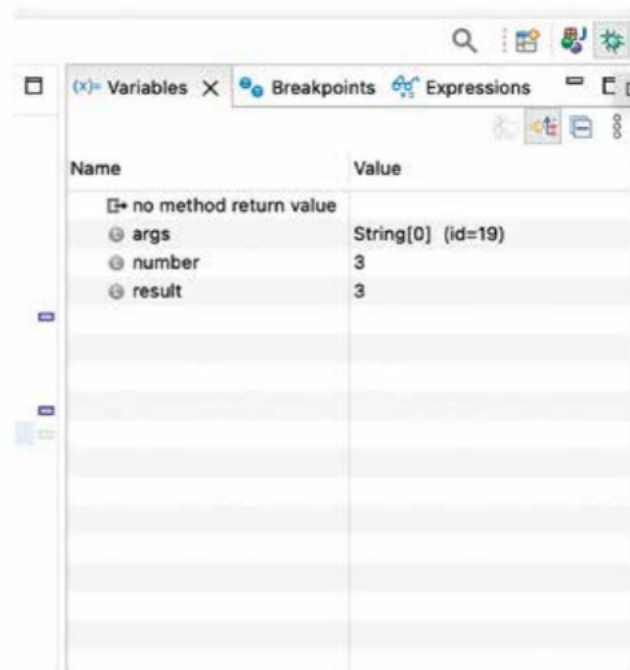
■ Breakpoint debugging in Eclipse

After setting up the breakpoints, the next step is to run the program in debugging mode. This is done by clicking on the Debug button (the one next to the Run button), which looks like a little bug, as shown in the image below.



■ Running the program in debugging mode in Eclipse

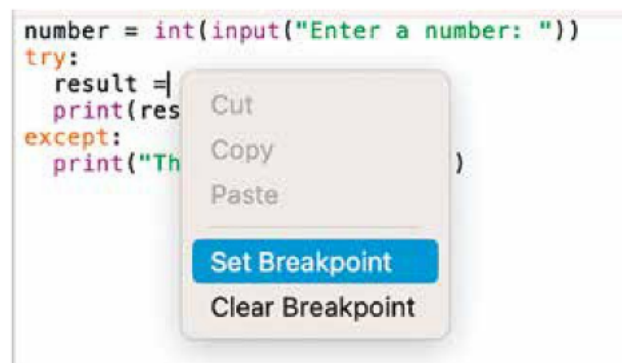
Next, you can see the Breakpoint window that shows the variables, breakpoints and expressions, as here:



■ Breakpoint window in Eclipse

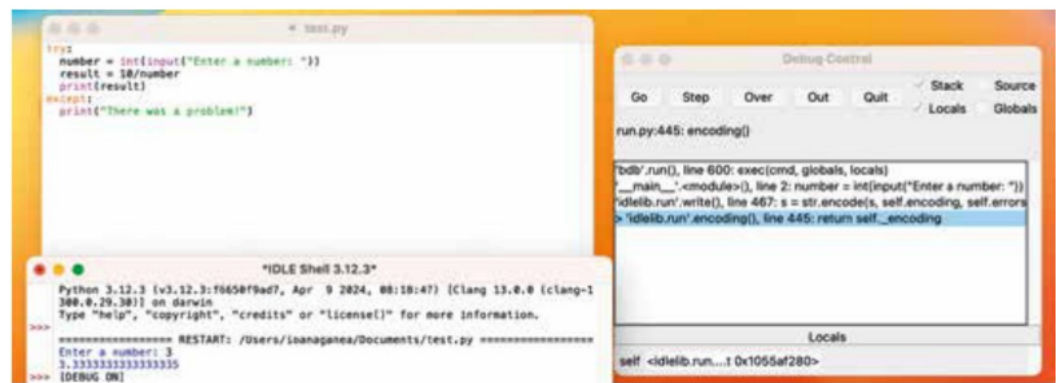
The code will execute as normal, but it will interrupt its execution the moment it reaches the set breakpoint.

In IDLE (Python), there is only a need to right-click on the line where you want the breakpoint to be set and choose the option Set Breakpoint.



■ Breakpoint debugging in IDLE

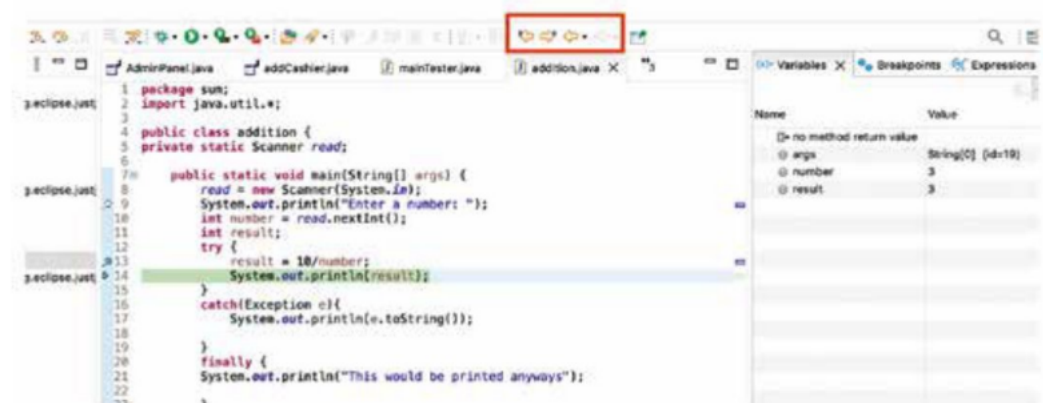
After that, you can simply run the script (F5). Once this has happened, you can press Debug on the menu and the debugger window will appear.



■ Debugger window in IDLE

■ Step-by-step code execution

To monitor and see what happens with the variables, you have to press Step into / Step over or use Step filters buttons. If you want to check what happens at the breakpoint line then you would choose the Step into button, but if you want to skip that line and execute the following one you would choose the Step over button. In Eclipse, those buttons are the arrows shown in the following screenshot:



■ Step-by-step execution in Eclipse

■ Print statements

When testing your code, you might ask yourself whether the execution of the program has reached a specific line of code, whether a variable changed its value as expected or whether a decision statement has been evaluated to true or false. Including print statements into your code to trace such changes is a useful method that would help you identify when exactly your code stopped running as expected and give you an idea of what went wrong. The only impediment might be that, once the debugging has occurred and the errors are fixed, you will have to delete those print statements.

● Key information

You need to be able to construct and use common debugging techniques. Being able to complete trace tables is an important technique that can be used to test the logic and functionality of an algorithm, to identify how the variables change throughout the running of the program and to identify the expected output of an algorithm.

REVIEW QUESTIONS

- 1 Define the terms “exception” and “exception handling”.
- 2 Identify three possible causes of an exception.
- 3 Outline three features that support the debugging of the code in an IDE.

ACTIVITY

Affective skills: Demonstrate persistence and perseverance. Don’t give up when you realize that coding becomes challenging – use debugging techniques to identify the errors, focus on understanding the errors report and try different approaches to solving problems.

● TOK

Language and meaning

We use different programming languages to code, each with its own syntax and grammar rules. English is not suitable to be used as a programming language because it is ambiguous and many expressions can be interpreted differently based on the context in which they are used.

Analyse the following statements and identify the possible meanings of each of them:

- Peter and Anna are married.
- A salesman visited every house in the area.
- Look at that dog with one eye.

What are the essential features of a computer language? Why is there a need for a fixed vocabulary, unambiguous meaning and consistent grammar and syntax?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B2.2.1 Compare static and dynamic data structures
- ▶ B2.2.2 Construct programs that apply arrays and lists
- ▶ B2.2.3 Explain the concept of a stack as a Last In First Out (LIFO) data structure
- ▶ B2.2.4 Explain the concept of a queue as a First In First Out (FIFO) data structure

B2.2.1 Static vs dynamic data structures

A data structure is a way of storing and organizing data, so it allows for its manipulation in an efficient way and attempts to reduce the time and space complexity. Arrays, lists, stacks, queues, binary trees and hash tables are all examples of data structures. Data structures can be **static** or **dynamic**, the two differing in terms of storage and access to their elements.

A static data structure has a predefined fixed size, a specific memory space being allocated to it, that will not change when running the program, and elements that are stored in contiguous memory locations. Elements in a static data structure can be either **directly accessed** or **sequentially accessed**.

A dynamic data structure has no predefined fixed size, it can grow or decrease at runtime and its elements are stored in memory locations that are chained together but are not necessarily contiguous. Elements in a dynamic data structure cannot be directly accessed.

◆ **Static data**

structure: a data structure with predefined fixed size and elements stored in contiguous memory locations.

◆ **Dynamic data**

structure: a data structure that can grow or decrease at runtime, with elements stored in memory locations that are chained together, but not necessarily contiguous.

◆ **Direct access:** a method of access where elements are directly retrieved by using their index (position).

◆ **Sequential access:** a method of access where elements are checked one after another, from the beginning to the end of the data structure.

Static data structure

Advantages

- Fast access time, as elements can be directly accessed via the index (position of the element in the data structure)
- No need to deal with possible overflow or underflow errors when adding or deleting elements as the size is fixed
- Easier to program; there is no need to check on its size at any time

Disadvantages

- Inefficient use of memory, as there is a need to know in advance the likely size of the structure to allocate sufficient memory
- Memory is allocated, regardless of whether or not it is needed
- Does not allow for flexibility, as its size is fixed
- Deletion of an element might create a vacant slot between two other elements, and changing the code to reuse this space might take time
- Insertion of elements between two other elements if there isn't a vacant space is time-consuming

Dynamic data structure

Advantages

- Allows for flexibility – it can shrink or grow as needed at runtime (suitable when the size of data is not known in advance or may change in time)
- Efficient use of memory; as it can resize itself, there is no memory waste
- Insertion and deletion operations are optimal concerning time and space complexity

Disadvantages

- The programmer needs to deal with overflow (the structure might exceed the memory limit) and underflow (it might attempt to delete an item from an already empty structure) situations when inserting or deleting elements
- Harder to program, as the programmer is required always to keep track of its size and data allocations